

Sonar equipment used in a submarine and modern networking? Of course, it's the humble *ping* command, which was named after the sound that a Sonar makes in a submarine. If you've seen *Hunt for Red October* you'll know :-). So, continuing the marine theme, *ping* is the first port of call when you have a network problem, and naturally, everyone knows how to use it to check if some host is up.

But is that all *ping* can do? Even in the normal run, there's important information. Figure 1 shows a typical *ping* output.

There's a very important number that *ping* shows, called the 'round trip time' (RTT). RTT is a measure of how close a host is to you, based on how long it takes a packet to go out and come back again. RTT on a LAN tends to be less than a millisecond, while 2-3 milliseconds (as in Figure 1) is more typical of a wireless network. RTTs on WAN links are more in the 200-600 millisecond range, reflecting the number of routers that they have to go through.

Ping can also get you clued into an unreliable connection, by showing a packet loss in the status line (the last line but one above). For instance, it might say "50 packets transmitted, 47 received, 6% packet loss, time 44754ms," which would indicate a pretty bad connection.

But this information only shows up at the end, after you kill *ping*. What if you want to keep the *ping* running for a while and continuously see how reliable the connection is? Well, you can watch the ICMP sequence number to make sure it increases exactly by one and doesn't skip a few, but that's too tedious to keep up for a long time. I mean, that's what computers are for, right—to do the tedious stuff? So can the humble *ping* command do anything more?

Turns out it can, and in a very imaginative and simple way! The command to use is *ping -f -i 1 host*. With the *-f* option, *ping* prints a "." for every outgoing packet and a back-space for every reply. Thus the number of dots on the display is the number of *ping* packets that have not yet been acknowledged by the remote side. A fast and reliable connection will not show you a single dot—every dot will be cancelled by a back-space well before the next dot appears, so the cursor sits on the left of the screen and nothing seems to be happening.

If you see the number of dots increasing gradually, you know there are packet losses happening on the link. It's actually a pretty cool display, but in order to see it, you have to test it against an unreliable server or an unreliable network. For most home users, the best way to do this is to use a laptop to *ping* a wireless router, and gradually move the laptop further and further away from the access point.

When you use *-f*, don't forget the 1-second interval flag ("*-i 1*"). Otherwise, you get what is called a 'flood

```
10:27:30 s1ta-mv:~$ ping 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=1.56 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=2.08 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=2.08 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=2.32 ms
64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=2.52 ms
^C
--- 10.0.0.1 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4477ms
rtt min/avg/max/mdev = 1.567/2.316/2.653/0.591 ms
10:27:37 s1ta-mv:~$
```

Figure 1: The output of a *ping* command

```
My tracemoute [v0.72]
afomali@tcn:~$ mtr 10.0.0.1
mtr: Wed: Display mode: Meters statistics: Order of fields: quit
Host: Loss: Last: Avg: Best: Worst: SDev:
1. 200.200.222.1 0.0% 0.0 0.0 0.0 0.0 0.0
2. 121.241.178.34 static-hyderabad.vsnl.net 0.0% 23.3 36.0 0.0 23.7 7.9
3. 112Pfr-203.189.131.70 static.vsnl.net 25.0% 23.5 0.0 2.0 23.5 42.0
4. 50.163.36.1 static.vsnl.net.in 80.0% 109.3 84.1 59.0 109.3 35.4
5. 50.163.36.1 static.vsnl.net.in 0.0% 29.0 63.0 29.0 29.0 59.2
6. 50.163.36.218 static.vsnl.net.in 0.0% 129.0 124.2 129.0 129.0 1.0
7. 50.163.36.209 static.vsnl.net.in 0.0% 89.5 96.0 89.5 129.3 29.2
8. 120.21.187.00 0.0% 533.3 525.5 517.0 533.5 1.0
9. 10.147.0.65 0.0% 117.0 121.0 115.5 130.5 0.0
10. 121.71.167.76 0.0% 149.7 156.0 129.0 149.7 6.7
```

Figure 2: An example *mtr* output

ping, which can look like a Denial of Service attack to the target host, and they might complain (or worse, retaliate). In fact, a fast machine on a fast network can bring down a network using a *flood ping* without an interval specified! However, if the target host is yours or you have permission to do so, it can be fun to try something like *ping -f -c 500 -s 1400 host*. The laptop + wireless method of simulating a flaky connection is really useful to see this in action. Also, try different values for the packet size (the *-s* option).

This is not just fun—you'll start to recognise that this simple dot pattern can clue you into troublesome connections very quickly, although once again, I must repeat that *-f* without *-i 1* should be used very carefully and sparingly, and only on your own hosts.

Who's dropping the ball?

So you have a flaky connection to your office network... and your VPN keeps dropping off. Or your YouTube feed is constantly stopping to buffer? I mean, we know which is more likely, right?

A *flood ping* tells you there are lots of dropped packets but doesn't tell you where or who's responsible. If you've ever done a *traceroute*, you know there are multiple hosts in between yourself and the target, and it may be useful to know where among these hops the packet loss is occurring.

This is what *mtr* shows you: it shows where the packet loss is happening, in real time, using ICMP ECHO requests (i.e., *ping* packets). It's one of the best tools for figuring out where the problems are, with a simple but really useful display, including a quick online help screen. The default screen looks like Figure 2, once it's started up, though, of course, it's continuously updating.

You can quickly see which intermediate router